Experiment No.: 2 Date: _____

Experiment Question

For a given set of training data examples stored in a .CSV file, implement and demonstrate the Candidate-Elimination algorithm in Python to output a description of the set of all hypotheses consistent with the training examples.

Aim

To implement the Candidate-Elimination algorithm in Python and output the specific boundary S and the general boundary G, which together describe the version space of all hypotheses consistent with the training examples in a .CSV file.

Concept

The version space is the set of all hypotheses consistent with the training examples. Instead of listing every hypothesis, Candidate-Elimination stores only its two boundaries: S, the maximally specific consistent hypotheses, and G, the maximally general consistent hypotheses. Every hypothesis between S and G is also consistent. A positive example generalises S and removes the members of G that do not cover it. A negative example specialises the members of G that wrongly cover it. Unlike FIND-S, it uses the negative examples and describes all consistent hypotheses, not just one. When S and G become equal, the concept has been learned exactly.

Dataset Description

The experiment has five sub-programs. Each uses its own small training set stored in a .CSV file (target value Yes / No); the datasets are given with the sub-programs:

2(a) EnjoySport (6 attributes)

2(b) Buy Mango (4 attributes)

2(c) Loan Approval (4 attributes)

2(d) Flu Diagnosis (4 attributes)

2(e) Play Outdoors (4 attributes)

Algorithm

1. Initialise G to the most general hypothesis <?, ?, ..., ?> and S to the first positive example.
2. For each positive example: remove from G every hypothesis that does not cover the example; generalise S minimally so that it covers the example.
3. For each negative example: for each hypothesis in G that covers it, replace it by its minimal specialisations that exclude the example and are still more general than S.
4. Remove from G any hypothesis that is less general than another hypothesis in G.
5. After all examples have been processed, output S and G.

Step-by-step Procedure

1. Import the csv module, load the .CSV file and find the possible values of each attribute.
2. Initialise G = [<?, ?, ..., ?>]; S is set to the first positive example.
3. For each row, update S and G according to whether the example is positive or negative.
4. Print S and G after every example to show how the boundaries move.
5. Print the final boundaries S and G.

Formulae

Version space: VS = { h in H | h is consistent with every training example D }

S = { s in VS | no other member of VS is more specific than s } G = { g in VS | no other member of VS is more general than g }

Every h with $s \leq h \leq g$ for some s in S and g in G belongs to VS.

Sub-program 2(a): EnjoySport

Dataset Description

4 training examples stored in enjoysport.csv: 6 input attributes and the target attribute **EnjoySport** (Yes / No). The boundaries S and G are computed after every example.

Sky	AirTemp	Humidity	Wind	Water	Forecast	EnjoySport
Sunny	Warm	Normal	Strong	Warm	Same	Yes
Sunny	Warm	High	Strong	Warm	Same	Yes
Rainy	Cold	High	Strong	Warm	Change	No
Sunny	Warm	High	Strong	Cool	Change	Yes

Python Program

```
import csv

with open("enjoysport.csv") as f:
    data = list(csv.reader(f))
header, rows = data[0], data[1:]
n = len(header) - 1
domains = [sorted({r[j] for r in rows}) for j in range(n)]
fmt = lambda h: "<" + ",".join(h) + ">"
# does hypothesis h cover x? (also: is h more general than x?)
covers = lambda h, x: all(a == "?" or a == b for a, b in zip(h, x))

S, G = None, [["?"] * n]            # specific and general boundaries
for i, row in enumerate(rows, 1):
    x, label = row[:-1], row[-1]
    if label == "Yes":                  # positive example
        G = [g for g in G if covers(g, x)]
        S = x.copy() if S is None else [s if s == v else "?" for s, v in zip(S, x)]
    else:                               # negative example
        newG = []
        for g in G:
            if not covers(g, x):
                newG.append(g)
                continue
            for j in range(n):          # minimal specialisations of g
                for v in domains[j]:
                    if g[j] == "?" and v != x[j] and S[j] == v:
                        h = g[:j] + [v] + g[j + 1:]
                        if h not in newG:
                            newG.append(h)
        G = [g for g in newG if not any(g != o and covers(o, g) for o in newG)]
    print(f"Example {i} ({label:3}):  S = {fmt(S)}")
    print(" " * 16 + "G = { " + ", ".join(fmt(g) for g in G) + " }")

print("\nFinal specific boundary S:", fmt(S))
print("Final general boundary G :", ", ".join(fmt(g) for g in G))
```

Fig. 2a.1 Program (exp2a_candidate_elimination.py)

Output

```
Example 1 (Yes):  S = <Sunny,Warm,Normal,Strong,Warm,Same>
                  G = { <?,?,?,?,?,?> }
Example 2 (Yes):  S = <Sunny,Warm,?,Strong,Warm,Same>
                  G = { <?,?,?,?,?,?> }
Example 3 (No ):  S = <Sunny,Warm,?,Strong,Warm,Same>
                  G = { <Sunny,?,?,?,?,?>, <?,Warm,?,?,?,?>, <?,?,?,?,?,Same> }
Example 4 (Yes):  S = <Sunny,Warm,?,Strong,?,?>
                  G = { <Sunny,?,?,?,?,?>, <?,Warm,?,?,?,?> }

Final specific boundary S: <Sunny,Warm,?,Strong,?,?>
Final general boundary G : <Sunny,?,?,?,?,?>, <?,Warm,?,?,?,?>
```

Fig. 2a.2 Output of the program

Result / Conclusion

After the four examples S = {<Sunny, Warm, ?, Strong, ?, ?>} and G = {<Sunny, ?, ?, ?, ?, ?>, <?, Warm, ?, ?, ?, ?>}. The hypothesis <?, ?, ?, ?, ?, Same> was removed from G by the fourth (positive) example. The version space contains six hypotheses between S and G.

Sub-program 2(b): Buy Mango

Dataset Description

5 training examples stored in mango.csv: 4 input attributes and the target attribute **BuyMango** (Yes / No). The boundaries S and G are computed after every example.

Colour	Size	Taste	Skin	BuyMango
Yellow	Big	Sweet	Smooth	Yes
Yellow	Small	Sweet	Smooth	Yes
Green	Big	Sour	Rough	No
Yellow	Big	Sweet	Rough	Yes
Red	Small	Sweet	Smooth	No

Python Program

```
import csv

with open("mango.csv") as f:
    data = list(csv.reader(f))
header, rows = data[0], data[1:]
n = len(header) - 1
domains = [sorted({r[j] for r in rows}) for j in range(n)]
fmt = lambda h: "<" + ",".join(h) + ">"
# does hypothesis h cover x? (also: is h more general than x?)
covers = lambda h, x: all(a == "?" or a == b for a, b in zip(h, x))

S, G = None, [["?"] * n]            # specific and general boundaries
for i, row in enumerate(rows, 1):
    x, label = row[:-1], row[-1]
    if label == "Yes":                  # positive example
        G = [g for g in G if covers(g, x)]
        S = x.copy() if S is None else [s if s == v else "?" for s, v in zip(S, x)]
    else:                               # negative example
        newG = []
        for g in G:
            if not covers(g, x):
                newG.append(g)
                continue
            for j in range(n):          # minimal specialisations of g
                for v in domains[j]:
                    if g[j] == "?" and v != x[j] and S[j] == v:
                        h = g[:j] + [v] + g[j + 1:]
                        if h not in newG:
                            newG.append(h)
        G = [g for g in newG if not any(g != o and covers(o, g) for o in newG)]
    print(f"Example {i} ({label:3}):  S = {fmt(S)}")
    print(" " * 16 + "G = { " + ", ".join(fmt(g) for g in G) + " }")

print("\nFinal specific boundary S:", fmt(S))
print("Final general boundary G :", ", ".join(fmt(g) for g in G))
```

Fig. 2b.1 Program (exp2b_candidate_elimination.py)

Output

```
Example 1 (Yes):  S = <Yellow,Big,Sweet,Smooth>
                  G = { <?,?,?,?> }
Example 2 (Yes):  S = <Yellow,?,Sweet,Smooth>
                  G = { <?,?,?,?> }
Example 3 (No ):  S = <Yellow,?,Sweet,Smooth>
                  G = { <Yellow,?,?,?>, <?,?,Sweet,?>, <?,?,?,Smooth> }
Example 4 (Yes):  S = <Yellow,?,Sweet,?>
                  G = { <Yellow,?,?,?>, <?,?,Sweet,?> }
Example 5 (No ):  S = <Yellow,?,Sweet,?>
                  G = { <Yellow,?,?,?> }

Final specific boundary S: <Yellow,?,Sweet,?>
Final general boundary G : <Yellow,?,?,?>
```

Fig. 2b.2 Output of the program

Result / Conclusion

The negative examples specialised G, and the positive examples generalised S. The final boundaries are S = {<Yellow, ?, Sweet, ?>} and G = {<Yellow, ?, ?, ?>}. S and G are not equal, so the version space still holds two hypotheses and more examples are needed to decide whether Taste matters.

Sub-program 2(c): Loan Approval

Dataset Description

5 training examples stored in loan.csv: 4 input attributes and the target attribute **Approved** (Yes / No). The boundaries S and G are computed after every example.

Income	Credit	Job	Age	Approved
High	Good	Yes	Young	Yes
Medium	Good	Yes	Old	Yes
Low	Poor	Yes	Young	No
High	Good	No	Old	No
Low	Good	Yes	Old	Yes

Python Program

```
import csv

with open("loan.csv") as f:
    data = list(csv.reader(f))
header, rows = data[0], data[1:]
n = len(header) - 1
domains = [sorted({r[j] for r in rows}) for j in range(n)]
fmt = lambda h: "<" + ",".join(h) + ">"
# does hypothesis h cover x? (also: is h more general than x?)
covers = lambda h, x: all(a == "?" or a == b for a, b in zip(h, x))

S, G = None, [["?"] * n]            # specific and general boundaries
for i, row in enumerate(rows, 1):
    x, label = row[:-1], row[-1]
    if label == "Yes":                  # positive example
        G = [g for g in G if covers(g, x)]
        S = x.copy() if S is None else [s if s == v else "?" for s, v in zip(S, x)]
    else:                               # negative example
        newG = []
        for g in G:
            if not covers(g, x):
                newG.append(g)
                continue
            for j in range(n):          # minimal specialisations of g
                for v in domains[j]:
                    if g[j] == "?" and v != x[j] and S[j] == v:
                        h = g[:j] + [v] + g[j + 1:]
                        if h not in newG:
                            newG.append(h)
        G = [g for g in newG if not any(g != o and covers(o, g) for o in newG)]
    print(f"Example {i} ({label:3}):  S = {fmt(S)}")
    print(" " * 16 + "G = { " + ", ".join(fmt(g) for g in G) + " }")

print("\nFinal specific boundary S:", fmt(S))
print("Final general boundary G :", ", ".join(fmt(g) for g in G))
```

Fig. 2c.1 Program (exp2c_candidate_elimination.py)

Output

```
Example 1 (Yes):  S = <High,Good,Yes,Young>
                  G = { <?,?,?,?> }
Example 2 (Yes):  S = <?,Good,Yes,?>
                  G = { <?,?,?,?> }
Example 3 (No ):  S = <?,Good,Yes,?>
                  G = { <?,Good,?,?> }
Example 4 (No ):  S = <?,Good,Yes,?>
                  G = { <?,Good,Yes,?> }
Example 5 (Yes):  S = <?,Good,Yes,?>
                  G = { <?,Good,Yes,?> }

Final specific boundary S: <?,Good,Yes,?>
Final general boundary G : <?,Good,Yes,?>
```

Fig. 2c.2 Output of the program

Result / Conclusion

The boundaries meet after the fourth example: S = G = {<?, Good, Yes, ?>}. The version space has only one hypothesis, so the concept (good credit and a job) is learned exactly.

Sub-program 2(d): Flu Diagnosis

Dataset Description

5 training examples stored in flu.csv: 4 input attributes and the target attribute **Flu** (Yes / No). The boundaries S and G are computed after every example.

Fever	Cough	Fatigue	Headache	Flu
Yes	Yes	Yes	Yes	Yes
Yes	Yes	No	Yes	Yes
No	Yes	Yes	No	No
Yes	No	Yes	Yes	No
Yes	Yes	Yes	No	Yes

Python Program

```
import csv

with open("flu.csv") as f:
    data = list(csv.reader(f))
header, rows = data[0], data[1:]
n = len(header) - 1
domains = [sorted({r[j] for r in rows}) for j in range(n)]
fmt = lambda h: "<" + ",".join(h) + ">"
# does hypothesis h cover x? (also: is h more general than x?)
covers = lambda h, x: all(a == "?" or a == b for a, b in zip(h, x))

S, G = None, [["?"] * n]            # specific and general boundaries
for i, row in enumerate(rows, 1):
    x, label = row[:-1], row[-1]
    if label == "Yes":                  # positive example
        G = [g for g in G if covers(g, x)]
        S = x.copy() if S is None else [s if s == v else "?" for s, v in zip(S, x)]
    else:                               # negative example
        newG = []
        for g in G:
            if not covers(g, x):
                newG.append(g)
                continue
            for j in range(n):          # minimal specialisations of g
                for v in domains[j]:
                    if g[j] == "?" and v != x[j] and S[j] == v:
                        h = g[:j] + [v] + g[j + 1:]
                        if h not in newG:
                            newG.append(h)
        G = [g for g in newG if not any(g != o and covers(o, g) for o in newG)]
    print(f"Example {i} ({label:3}):  S = {fmt(S)}")
    print(" " * 16 + "G = { " + ", ".join(fmt(g) for g in G) + " }")

print("\nFinal specific boundary S:", fmt(S))
print("Final general boundary G :", ", ".join(fmt(g) for g in G))
```

Fig. 2d.1 Program (exp2d_candidate_elimination.py)

Output

```
Example 1 (Yes):  S = <Yes,Yes,Yes,Yes>
                  G = { <?,?,?,?> }
Example 2 (Yes):  S = <Yes,Yes,?,Yes>
                  G = { <?,?,?,?> }
Example 3 (No ):  S = <Yes,Yes,?,Yes>
                  G = { <Yes,?,?,?>, <?,?,?,Yes> }
Example 4 (No ):  S = <Yes,Yes,?,Yes>
                  G = { <Yes,Yes,?,?>, <?,Yes,?,Yes> }
Example 5 (Yes):  S = <Yes,Yes,?,?>
                  G = { <Yes,Yes,?,?> }

Final specific boundary S: <Yes,Yes,?,?>
Final general boundary G : <Yes,Yes,?,?>
```

Fig. 2d.2 Output of the program

Result / Conclusion

The boundaries meet after the fifth example: S = G = {<Yes, Yes, ?, ?>}. The hypothesis <?, Yes, ?, Yes> was removed from G by the last positive example. The version space has only one hypothesis: fever and cough.

Sub-program 2(e): Play Outdoors

Dataset Description

5 training examples stored in play.csv: 4 input attributes and the target attribute **Play** (Yes / No). The boundaries S and G are computed after every example.

Outlook	Temp	Humidity	Wind	Play
Overcast	Hot	High	Weak	Yes
Overcast	Mild	Normal	Weak	Yes
Sunny	Hot	High	Weak	No
Overcast	Cool	Normal	Strong	No
Overcast	Mild	High	Weak	Yes

Python Program

```
import csv

with open("play.csv") as f:
    data = list(csv.reader(f))
header, rows = data[0], data[1:]
n = len(header) - 1
domains = [sorted({r[j] for r in rows}) for j in range(n)]
fmt = lambda h: "<" + ",".join(h) + ">"
# does hypothesis h cover x? (also: is h more general than x?)
covers = lambda h, x: all(a == "?" or a == b for a, b in zip(h, x))

S, G = None, [["?"] * n]            # specific and general boundaries
for i, row in enumerate(rows, 1):
    x, label = row[:-1], row[-1]
    if label == "Yes":                  # positive example
        G = [g for g in G if covers(g, x)]
        S = x.copy() if S is None else [s if s == v else "?" for s, v in zip(S, x)]
    else:                               # negative example
        newG = []
        for g in G:
            if not covers(g, x):
                newG.append(g)
                continue
            for j in range(n):          # minimal specialisations of g
                for v in domains[j]:
                    if g[j] == "?" and v != x[j] and S[j] == v:
                        h = g[:j] + [v] + g[j + 1:]
                        if h not in newG:
                            newG.append(h)
        G = [g for g in newG if not any(g != o and covers(o, g) for o in newG)]
    print(f"Example {i} ({label:3}):  S = {fmt(S)}")
    print(" " * 16 + "G = { " + ", ".join(fmt(g) for g in G) + " }")

print("\nFinal specific boundary S:", fmt(S))
print("Final general boundary G :", ", ".join(fmt(g) for g in G))
```

Fig. 2e.1 Program (exp2e_candidate_elimination.py)

Output

```
Example 1 (Yes):  S = <Overcast,Hot,High,Weak>
                  G = { <?,?,?,?> }
Example 2 (Yes):  S = <Overcast,?,?,Weak>
                  G = { <?,?,?,?> }
Example 3 (No ):  S = <Overcast,?,?,Weak>
                  G = { <Overcast,?,?,?> }
Example 4 (No ):  S = <Overcast,?,?,Weak>
                  G = { <Overcast,?,?,Weak> }
Example 5 (Yes):  S = <Overcast,?,?,Weak>
                  G = { <Overcast,?,?,Weak> }

Final specific boundary S: <Overcast,?,?,Weak>
Final general boundary G : <Overcast,?,?,Weak>
```

Fig. 2e.2 Output of the program

Result / Conclusion

The boundaries meet after the fourth example: S = G = {<Overcast, ?, ?, Weak>}. The version space has only one hypothesis, so the concept (overcast sky and weak wind) is learned exactly.